MACHINE
LEARNING

机器学习建模基础——从概念到公司破产预测实战零代码做机器学习

从概念到公司破产预测实战

Day 2 上午 • 懂概念 • 辨范式 • 驾驭 AI

今天上午直播课 • 懂概念 • 辨范式 • 驾驭 AI

主讲：杨楷

kindyang.github.io

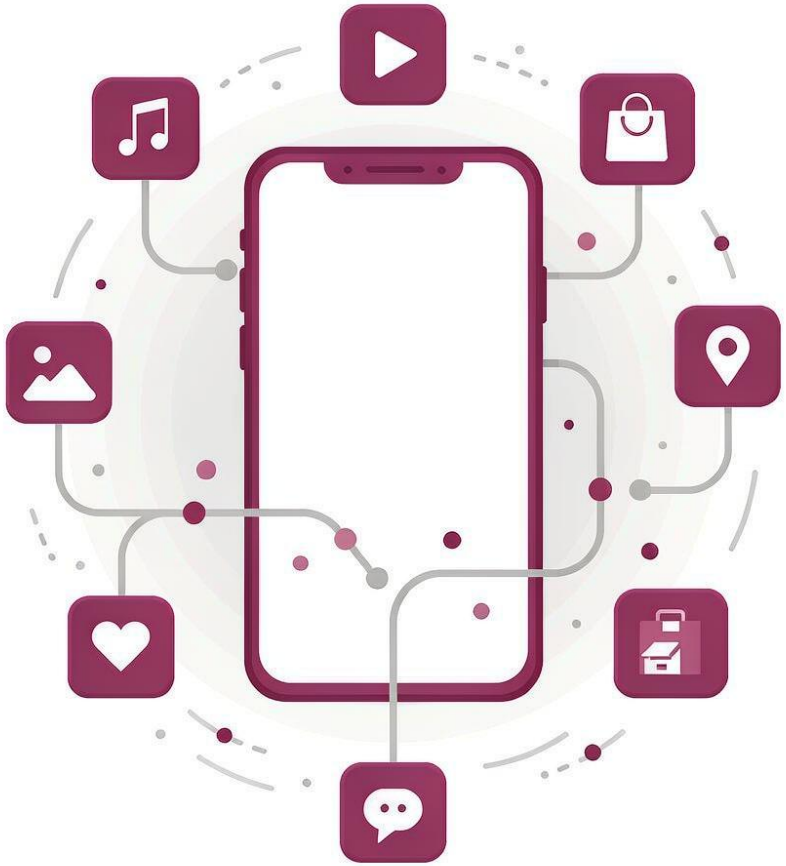
00

开场

What & Why

破冰：今天你已经"被机器学习"了多少次？

- **抖音推荐、打车 ETA、外卖调度、支付风控——每一次"好方便"，背后都是一次机器学习**
- 导航选路、扫脸支付、语音助手、垃圾邮件过滤
- 智能家居、就医挂号、购物『猜你喜欢』
- 你每天至少被几十个模型服务过——只是你没察觉



今天的承诺：你不需要写一行代码

你只需要会三件事，跟代码无关：

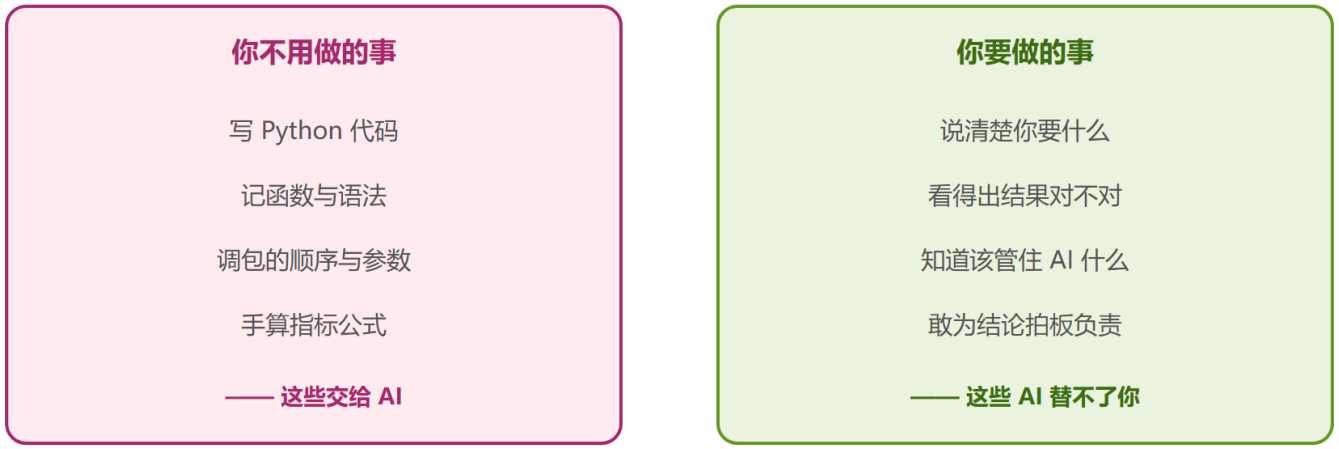
- 说清楚你要什么 —— 把需求讲成 AI 能执行的任务
- 看得出结果对不对 —— 知道该在哪些地方设卡
- 敢为结论拍板 —— 代价与标准由你定

代码由 AI 写，你负责判断它写得对不对

类比：不必会造汽车，但要会开车、看得懂仪表盘、知道何时踩刹车

不写代码，但要比写代码的人更懂流程

不写代码，但要比写代码的人更懂流程



类比：不必会造汽车，但要会开车、看得懂仪表盘、知道何时踩刹车

本次直播课路线图

今天分两段走：

- 前半场：机器学习是什么、有哪几种、具体流程是什么
- 后半场：用 AI Agent 从零做完破产预测，并固化成可复现流程

为什么先讲原理，而不是直接上手？

- 因为后半场你要给 AI 下指令、定规矩
- 不懂原理，你连该管什么都不知道 —— 更谈不上判断它对不对
- **全程不写代码，但你得比写代码的人更懂流程**

一句话：前半场给你判断力，后半场给你执行力

01

什么是机器学习

定义 + 3 个真实例子

从一个场景说起：AI 是怎么"学会"的？

从经验 E 学，完成任务 T，用性能 P 衡量（Mitchell，1997）

传统程序 vs 机器学习

传统程序：人写规则

if 负债率 > 90% : 高风险

elif 连续两年亏损 : 高风险

规则要人一条条想清楚、写全

现实：组合太多，人写不完

机器学习：数据里找规则

历史数据（特征）

历史结局（标签）

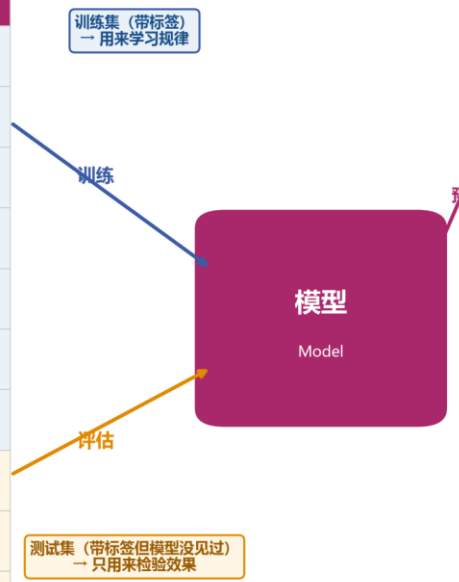
模型自己学出规则

人负责：给数据、定目标、评好坏

例子 1（有监督）：信用卡欺诈检测

- 历史交易 + 已标记结果（欺诈 / 正常）→ 训练模型
- 新交易进来→ 实时打分，异常则拦截
- 这类数据自带"标准答案"，叫有监督学习

客户ID	年龄	年收入(万)	月均消费(元)	近6月逾期次数	信用额度使用率(%)	是否违约(label)
C001	34	18	3200	0	22	0
C002	45	25	5600	2	78	1
C003	28	12	1800	0	35	0
C004	52	40	8900	3	91	1
C005	39	22	4100	1	58	0
C006	47	30	7200	4	88	1
C007	31	15	2600	0	41	0
C008	58	35	9600	5	95	1
C009	36	20	3700	1	52	0
C010	43	28	6800	3	84	1



新客户（无标签）

客户ID	年龄	年收入(万)	月均消费(元)	近6月逾期次数	信用额度使用率(%)	是否违约
N001	41	26	6100	2	81	?
N002	33	17	2400	0	38	?

预测

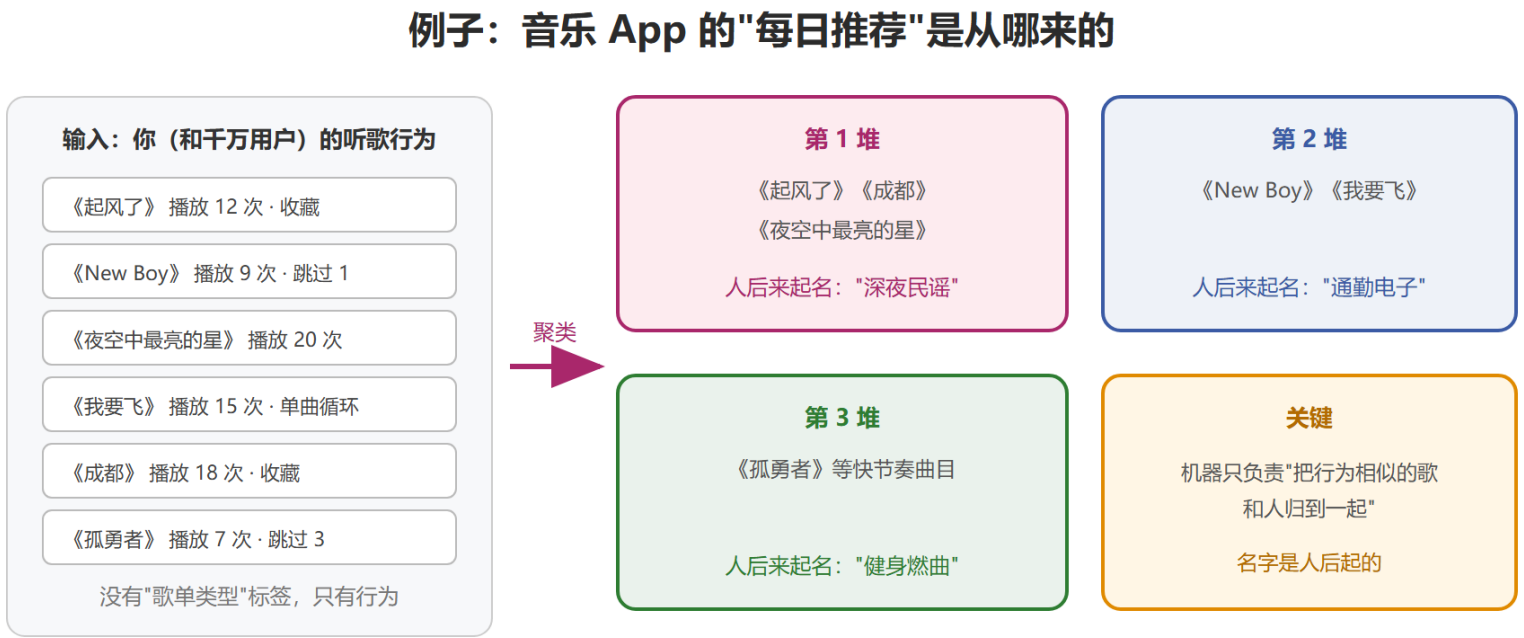
预测① P(违约)=0.87 → 判定 1

预测② P(违约)=0.12 → 判定 0

示例数据（虚构）

例子 2（无监督）：音乐 App 的"每日推荐"是从哪来的

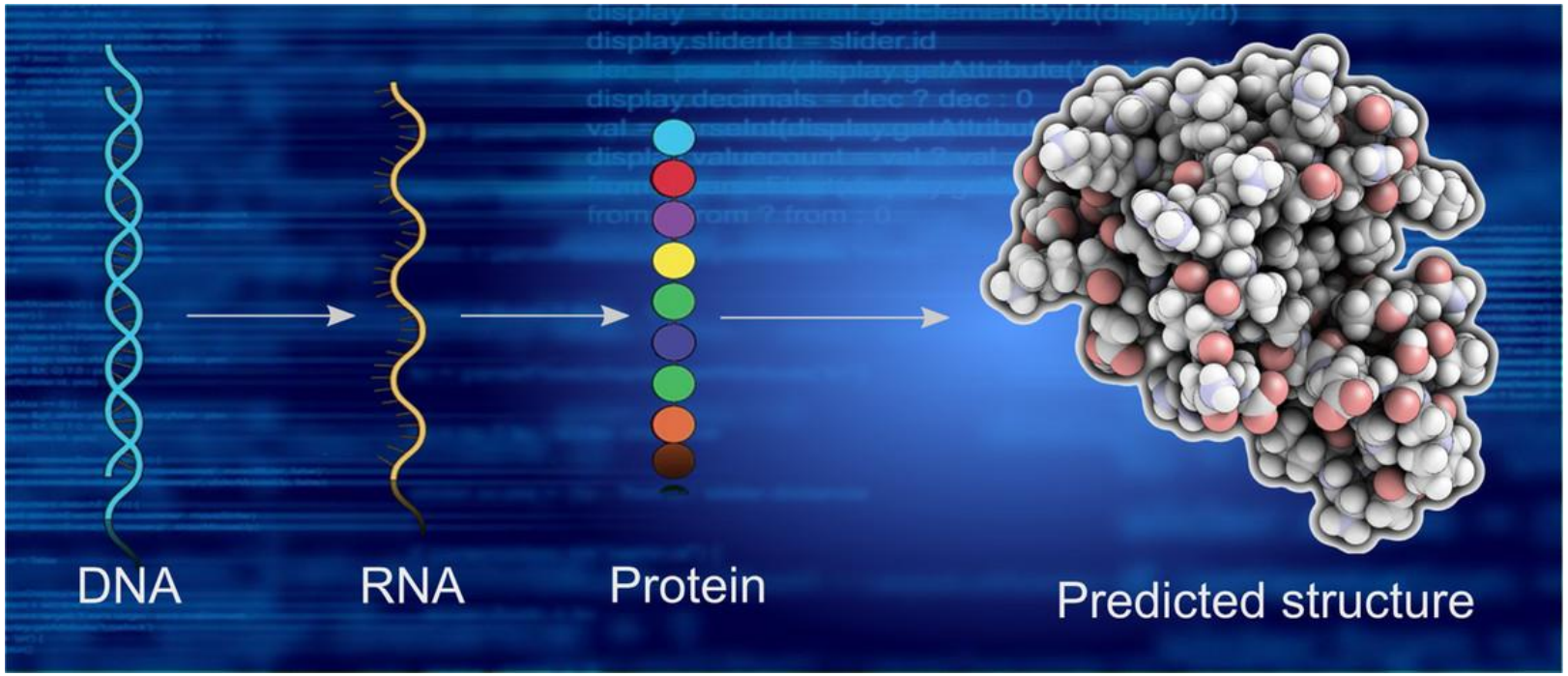
- **输入：你（和千万用户）的听歌行为——播放、跳过、收藏、单曲循环**
- 没有任何"歌单类型"标签
- 机器把"听歌行为相似"的歌与人自动归成几堆
- "深夜民谣""通勤电子"这些名字，是人后来才起的



"每日推荐"不是人编的歌单——**是机器从听歌行为里"聚"出来的**

例子 3（有监督）：AlphaFold 与科学发现

- **2021 年，AlphaFold 从基因序列预测蛋白质三维结构**
- 几十年积累的蛋白数据库 = 经验 E
- 机器学习不只是"猜你喜欢"，它已能推进科学前沿
- 预测目标：输入氨基酸序列 → 输出蛋白质每个原子的三维坐标



小结：三个例子的差别在于"有没有答案"

例子	有没有标准答案	属于哪一类
音乐"每日推荐"歌单	完全没有	无监督 • 聚类
信用卡欺诈检测	有（欺诈/正常）	有监督 • 分类
AlphaFold 预测结构	有（已知结构）	有监督 • 回归/结构预测

02

三大学习范式

有监督 / 无监督 / 半监督

总览：按"有没有标签"划分

有监督＝有老师批改；

无监督＝自己找规律；

半监督＝有限的练习题

机器学习的主要学习范式（按"有没有标签"划分）

有监督学习

全部数据都有标签（标准答案）

有 有 有 有 有

分类 · 回归

例：舞弊检测、房价预测

无监督学习

全部数据都没有标签

? ? ? ? ?

聚类 · 关联规则 · 降维

例：客户细分、啤酒与尿布

半监督学习

少量有标签 + 大量无标签

有 ? ? ? ?

标签稀缺场景

例：医学影像、财务舞弊

其他学习种类

强化学习

试错 + 奖励（AlphaGo）

自监督学习

从数据自身造标签（GPT）

深度学习

多层神经网络自动提特征

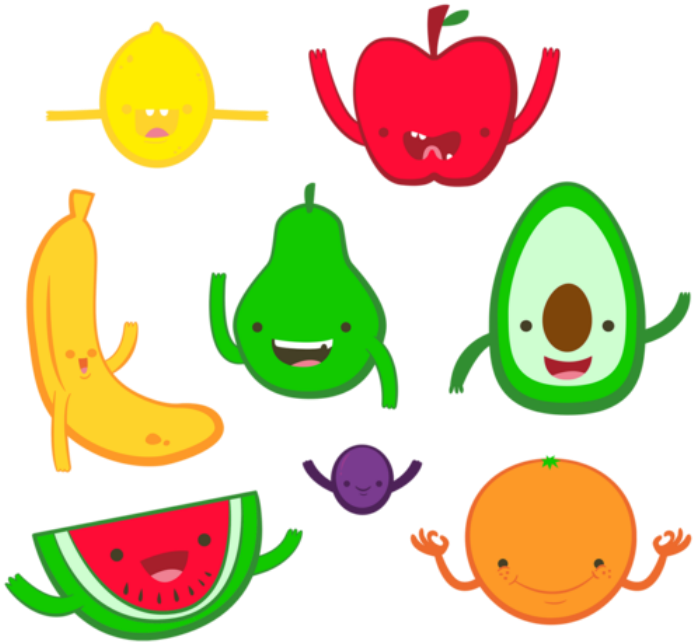
集成 / 迁移 / 在线

组合模型 · 知识复用

它们与三大范式往往组合使用：如"自监督预训练 + 有监督微调"

有监督学习①：什么是分类——水果分拣线

- **案例：一条水果智能分拣线，摄像头拍下每个水果的大小、颜色、瑕疵**
- 任务：机器给每个水果贴等级标签（特级 / 一级 / 次品）
- 标签来自质检员——这就是"有标签数据"，贵就贵在人工
- 数据集如何构建？



有监督学习②：二分类与多分类

- **同一条分拣线，换个标签定义就换了一个任务：**
- 二分类：能上架 / 不能上架（2 个类别）
- 多分类：特级 / 一级 / 二级 / 次品（4 个类别）
- 今天上午的破产预测，就是二分类



有监督学习③：训练集与测试集——"备考"的比喻

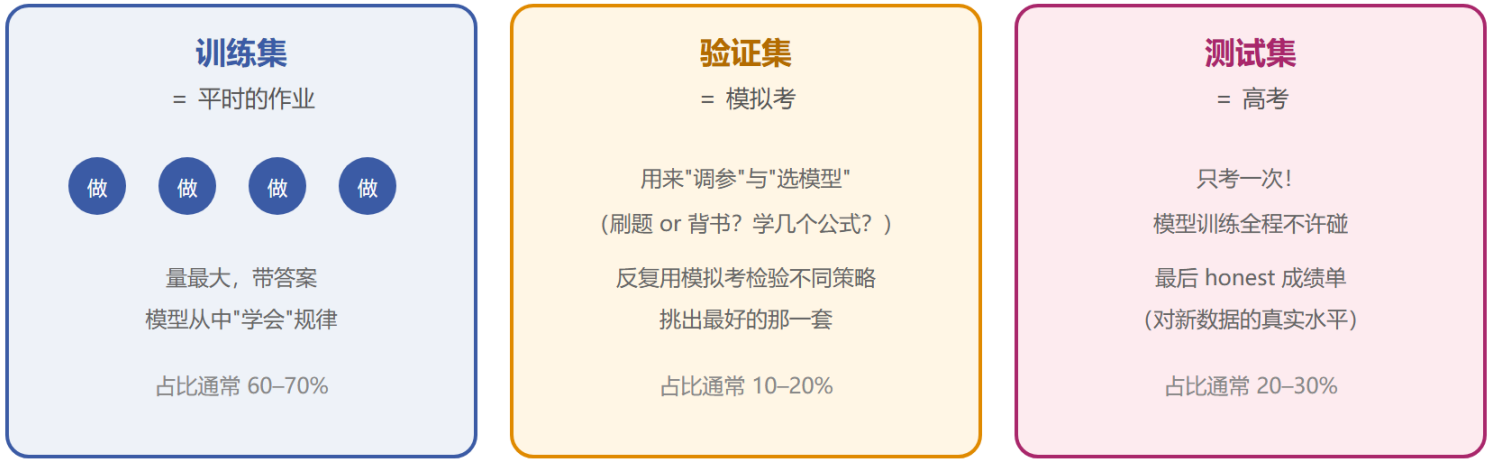
- **训练集（约 70%）＝平时作业：从中学会规律**
- 测试集（约 30%）＝高考：没见过的题，检验真实水平
- 大忌：拿测试集反复练习再自估分数 → 成绩虚高（数据泄漏）





有监督学习④：调参与验证集——为什么需要"模拟考"

- **模型自带"旋钮"（超参数）：树允许多深？森林用几棵树？**
- 不能用测试集调（那是高考），也不能只看作业分（会"背作业"＝过拟合）
- 验证集（约 10 - 20%）＝模拟考：每调一组旋钮，模拟考比一次
- 实操常用 K 折交叉验证：轮流把训练集的每一份当模拟考
- 口诀：作业练本事，模拟考选策略，高考定成绩



大忌：拿"高考真题"反复练习再自估分数 → 数据泄漏，成绩虚高

图：训练/验证/测试备考比喻（自绘）

有监督学习④：调参与验证集——为什么需要"模拟考"

- **模型自带"旋钮"（超参数）：树允许多深？森林用几棵树？**
- 不能用测试集调（那是高考），也不能只看作业分（会"背作业"＝过拟合）
- 验证集（约 10 - 20%）＝模拟考：每调一组旋钮，模拟考比一次
- 实操常用 K 折交叉验证：轮流把训练集的每一份当模拟考
- 口诀：作业练本事，模拟考选策略，高考定成绩

K 折交叉验证：轮流把每一份都当一次"模拟考"



有监督学习⑤：回归——预测连续的数

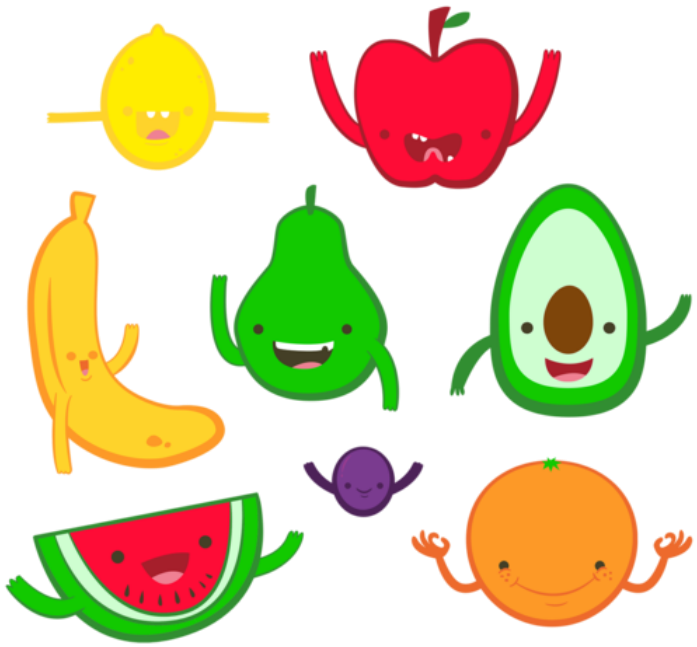
- **同一条分拣线，任务换成预测糖度 12.3%、重量 182 克**
- 离散的等级 → 分类；连续的数字 → 回归
- （分类 vs 回归，只差在标签长什么样）



无监督学习①：没有标签怎么学

- **分拣线的新一天：质检员还没定标准，一整车水果等着分**

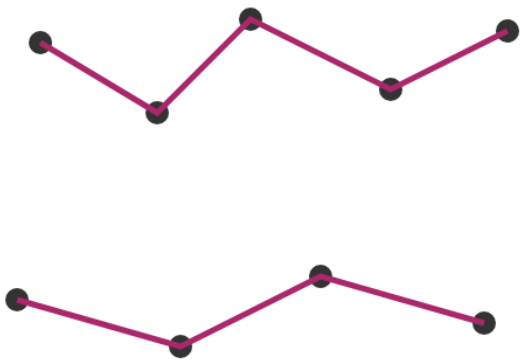
- 机器按"长得像"自动归堆：大而红的一堆、小而青的一堆
- 没有对错评分——只有"分组好不好解释"
- 类比：有监督 = 父母指着水果教孩子名字（有人给答案）
- 无监督 = 早期人类在野外观察果子，把相似的归成一堆，再给一个代号
- ——给『这一堆』起名字，就是最早的语言，也是聚类的本质



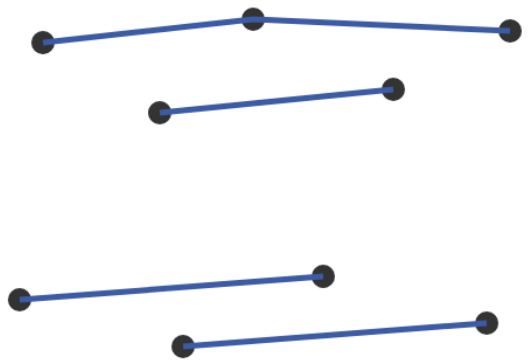
无监督学习②：星座的启示

- 星星本来没有标签，是人把它们"连"成了星座
- 同一片星空，连法不同，星座就不同

星座的启示：同一片星空，连法不同，星座就不同



按"距离远近"连线 → 分成 2 组



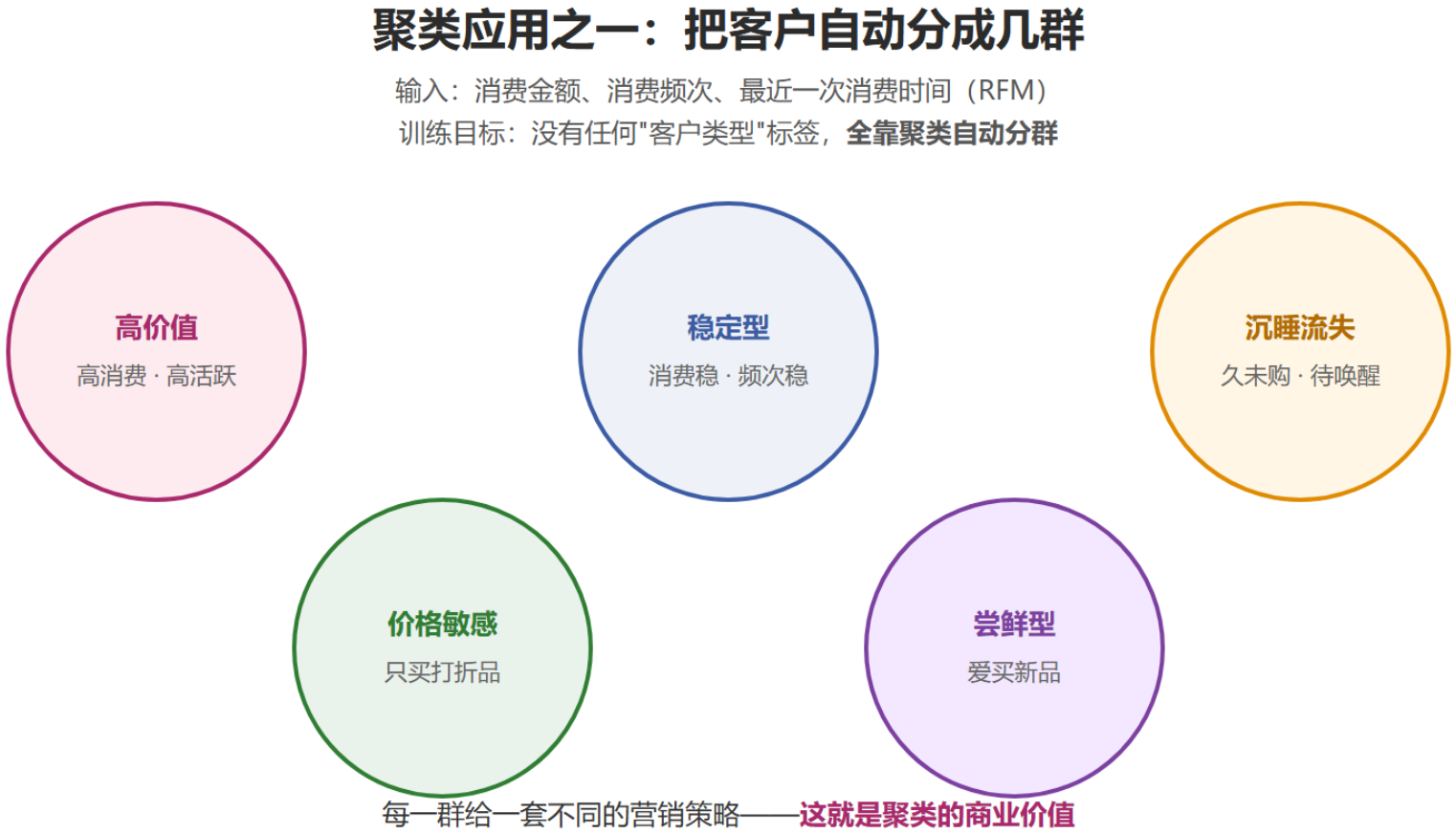
按"上方/下方 + 亮度"连线 → 分成 4 组

星星本身没有任何标签——**分组方式取决于你选的"相似标准"**

聚类没有"正确答案"，只有"能不能解释业务"

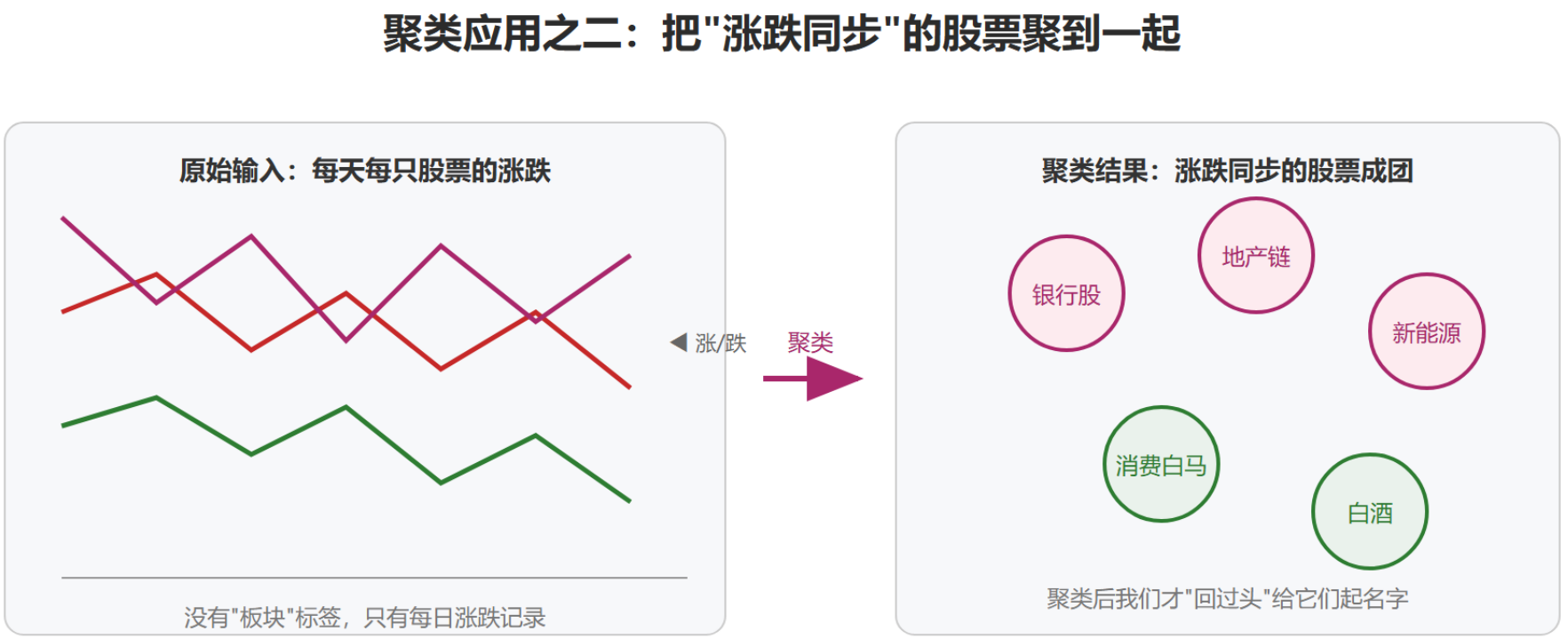
无监督学习③：聚类应用——客户自动分群

- 输入：消费金额 / 频次 / 最近消费时间，没有"客户类型"标签
- 每群给一套不同的营销策略



无监督学习④：聚类应用——涨跌同步的股票

- **输入：每日涨跌记录；输出：涨跌同步的股票自动成团**
- 用途：组合分散风险，发现"看起来无关其实同涨同跌"的隐性板块



用途：组合分散风险 · 发现"看起来无关其实同涨同跌"的隐性板块

无监督学习⑤：关联规则——啤酒与尿布

- 从购物篮里发现"一起买"的组合，同样不需要任何标签
- 经典案例：啤酒与尿布（传说真实性存疑，但方法千真万确）



半监督学习①：为什么需要它

- **打标签很贵：破产要等法院裁定；财报却几乎免费可得**
- 半监督的赌注：用海量便宜的无标签数据，放大少量昂贵标签的价值

半监督学习：少量标签 + 大量无标签

有标签样本（贵，很少）

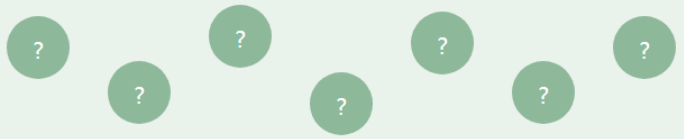
需要人工审计 / 监管处罚公告确认



本案：4,943 条样本中仅 28 条舞弊（0.6%）

无标签样本（便宜，海量）

财报公开可得，但无人逐一核查真伪

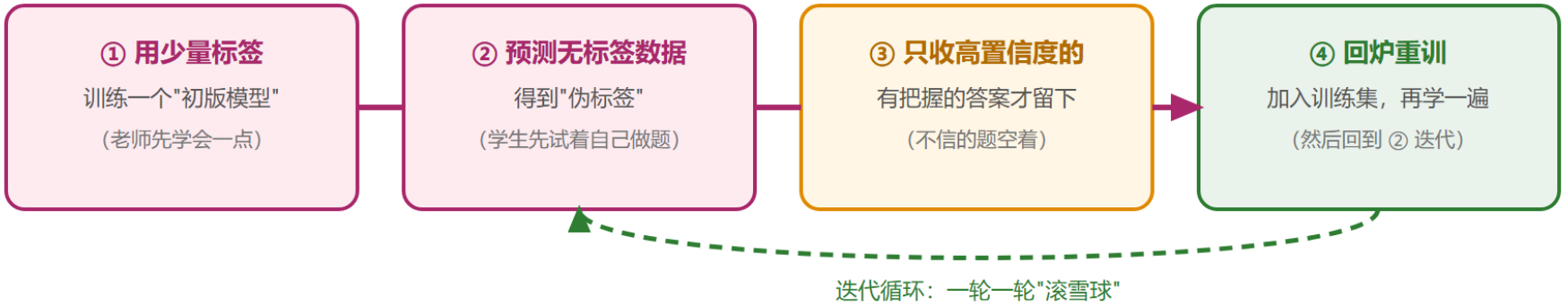


利用无标签数据的结构，放大少量标签的价值

典型思路：先用有标签训练 → 对无标签"伪标注"高置信样本 → 再回头重训练

半监督学习②：自训练 / 伪标签（主流思路，但不是唯一思路）

- **四步循环：少量标签训练初版模型 → 给无标签数据打伪标签 → 只收高置信度的 → 回炉重训，反复迭代**
- 这是半监督最主流、最好落地的思路，但不是唯一思路
- 其他思路（本课不展开）：协同训练、图标签传播、半监督 SVM、EM 生成式方法
- 最大风险：确认偏差——错的伪标签会被反复自我确认，必须设高阈值、多轮校验
- 参考：Yarowsky（1995）思想源头；Lee（2013）深度学习伪标签；FixMatch（Sohn et al.，2020）现代代表



注意：这是"主流思路"，但不是"唯一思路"

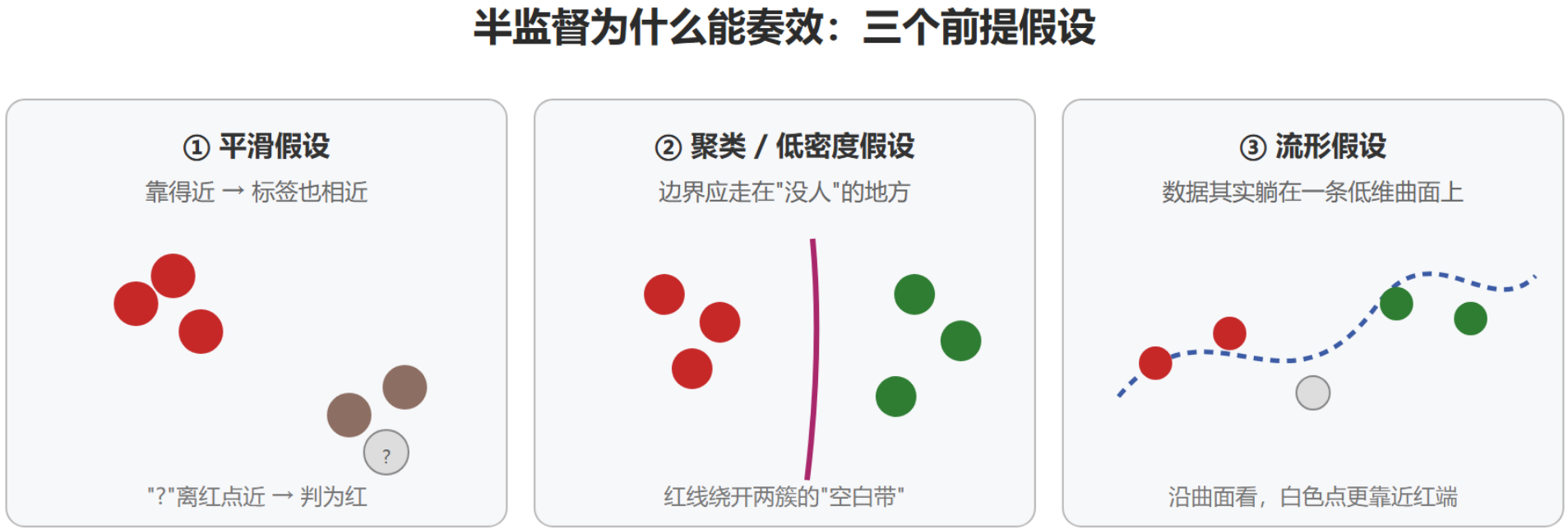
其他半监督思路还有：协同训练（多个模型互相教）· 图标签传播（标签沿相似网络扩散）· 低密度分离（如半监督 SVM）· 生成式方法（如 EM）

共同风险：确认偏差——错误的伪标签会被反复"自我确认"，所以必须设高阈值、多轮校验

自训练思想源头：Yarowsky (1995)；深度学习版伪标签：Lee (2013)；现代代表：FixMatch (Sohn et al., 2020)

半监督学习③：它凭什么能奏效

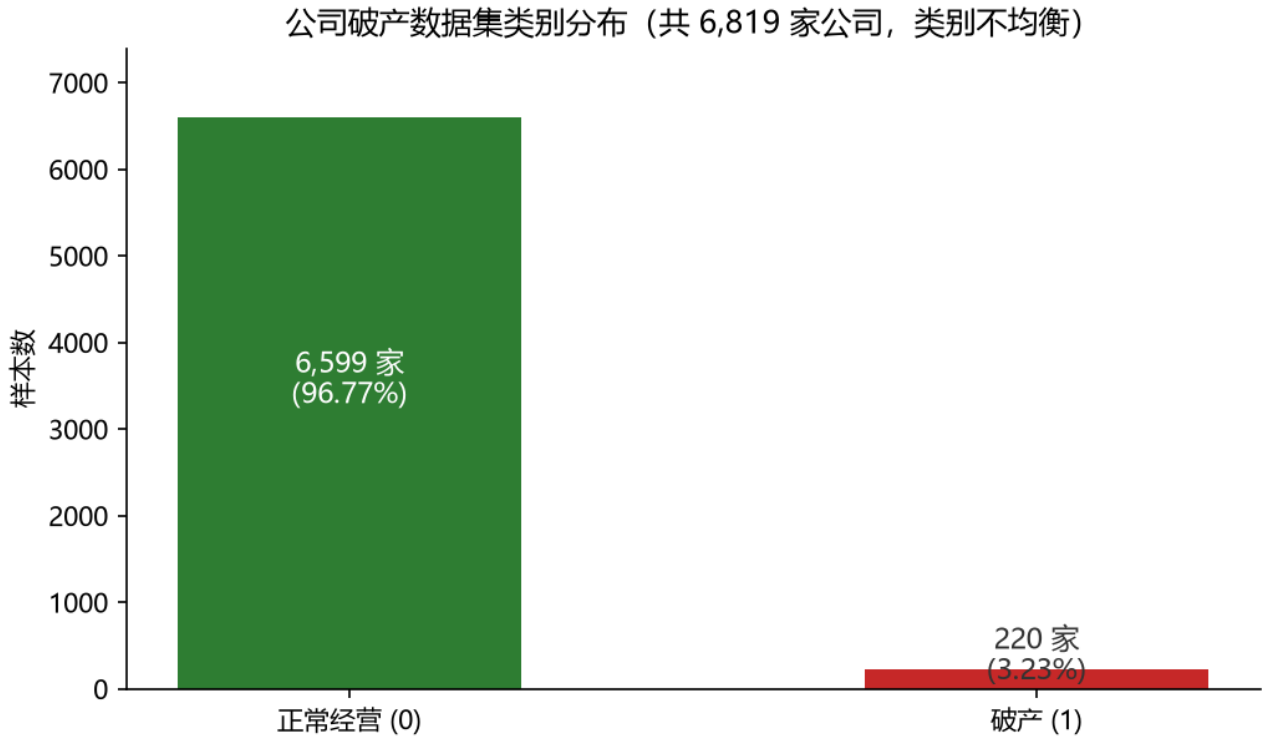
- **三大假设（没"看全局的前提"，半监督不成立）：**
- 平滑假设：相近的样本，标签也应相近
- 聚类/低密度假设：同类成簇，决策边界应走在数据稀疏处
- 流形假设：数据其实躺在低维曲面上



假设不成立时，硬上半监督可能**越学越差**——这正是"确认偏差"的根源（Chapelle et al., 2006）

半监督学习④：回到我们的破产场景

- **3.23% 的破产标签 = 典型的"少量贵标签"场景**
- 今天上午仍用有监督二分类做演示（好上手）；课后延伸可尝试自训练



三大范式对比小结

	有监督	无监督	半监督
标签	全有	全无	少量有
典型任务	分类、回归	聚类、关联规则	标签稀缺的分类
例子	破产预测、房价	客户分群、啤酒尿布	医学影像
成本	打标签贵	数据便宜	兼而有之

先记下来：等会儿管 AI，全靠这几条

前面学到的原理	等会儿要写的约束
训练集 / 测试集必须分开	锁死 test.csv，训练步不许碰
调参只能用验证集	5 折交叉验证，测试集不参与
准确率在不均衡下会骗人	不许只看准确率，看召回 / F1 / AUC
漏报比误报贵得多	**只能你定：AI 不懂你的业务代价**
指标口径要全程统一	写进 README，每一步都照它执行

不知道原理，你连该管什么都写不出来

不知道原理，你连该管什么都写不出来

03

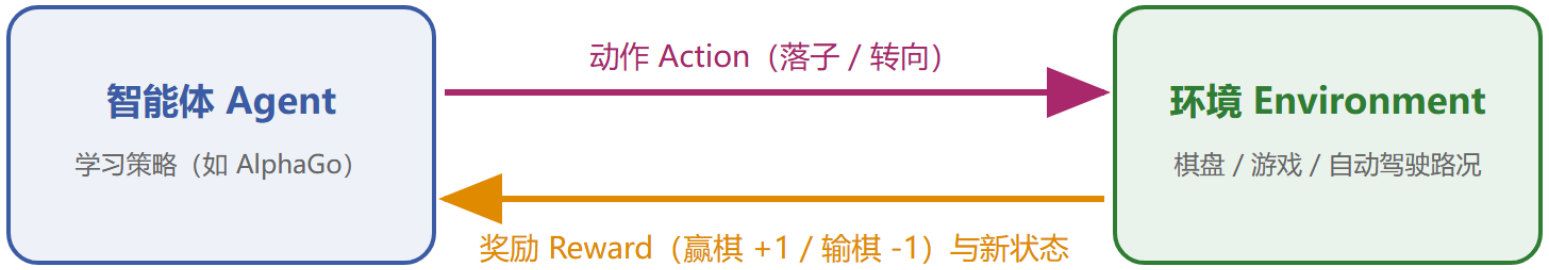
其他学习种类

强化 • 自监督 • 深度学习

强化学习：试错 + 奖励

- 没有答案标签，只有延迟的奖励信号（赢/输）
- 代表：AlphaGo 自我对弈、自动驾驶
- 金融应用：算法交易执行、投资组合再平衡、做市报价、信贷定价

强化学习：智能体通过"试错 + 奖励"学会决策



没有标准答案标签，只有延迟的奖励信号 —— 与有监督学习最大的区别

自监督学习：数据自己给自己当老师

- 把句子挖空让模型填 ➜ 标签是自己"造"出来的
- GPT 的"预测下一个词"、BERT 的"掩码语言模型"都是如此



深度学习与表示学习

- 多层神经网络自动提特征，不用人写"负债率"这类公式
- 它不是第四种范式，而是一种模型结构，可服务上述任何一种学习



它不是第四种"范式"——**它是一种模型结构**，可以服务于有监督 / 自监督 / 强化学习

全家福：一张表记住所有学习种类

学习种类	一句话记忆
有监督	有老师批改作业
无监督	自己找规律
半监督	少数老师带大量学生
强化	试错中成长
自监督	做自己的练习册
深度学习	多层网络自动提特征
集成学习	三个臭皮匠

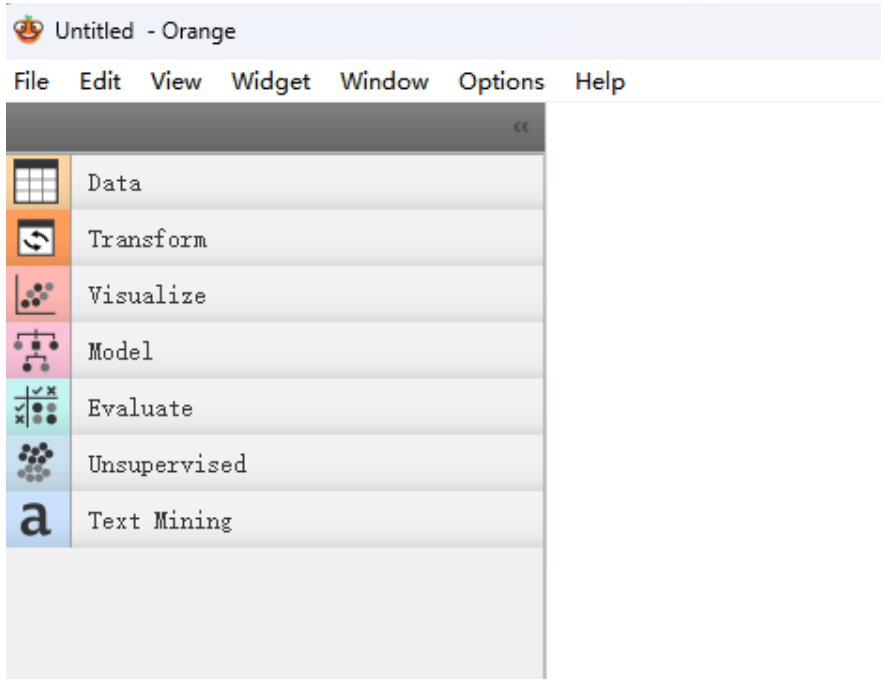
03+

认识 Orange：零代码工作台

拖拽连线 • 训练评估 • 一条龙

Orange 是什么

- **开源、免费的数据挖掘与机器学习工作台**
- C++ 底层引擎 + Python 接口，拖拽式连线搭建流程
- 不用写代码：数据导入 → 预处理 → 建模 → 评估，全在画布上完成
- 与 WorkBuddy 对照：Orange 你亲手连线，WorkBuddy 你指挥 AI

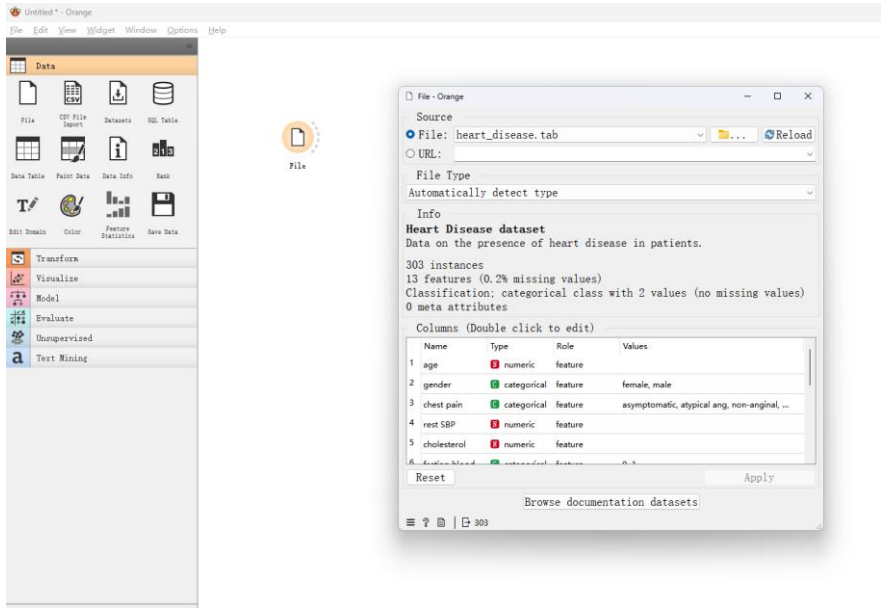


认识界面：六大模块一次看全

模块	干什么	本课用在哪
数据导入（File）	读入 Excel / CSV，查看数据	导入破产 / 舞弊数据
数据预处理（Preprocess）	缺失值、标准化、特征选择	数据准备
数据可视化	直方图、箱线图、散点图	探索数据
机器学习	Tree / Forest / k-Means 等	下午建模主角
模型评估（Prediction）	测试集打分、混淆矩阵	评估
文本挖掘	需手动扩展模块	Day 3 用

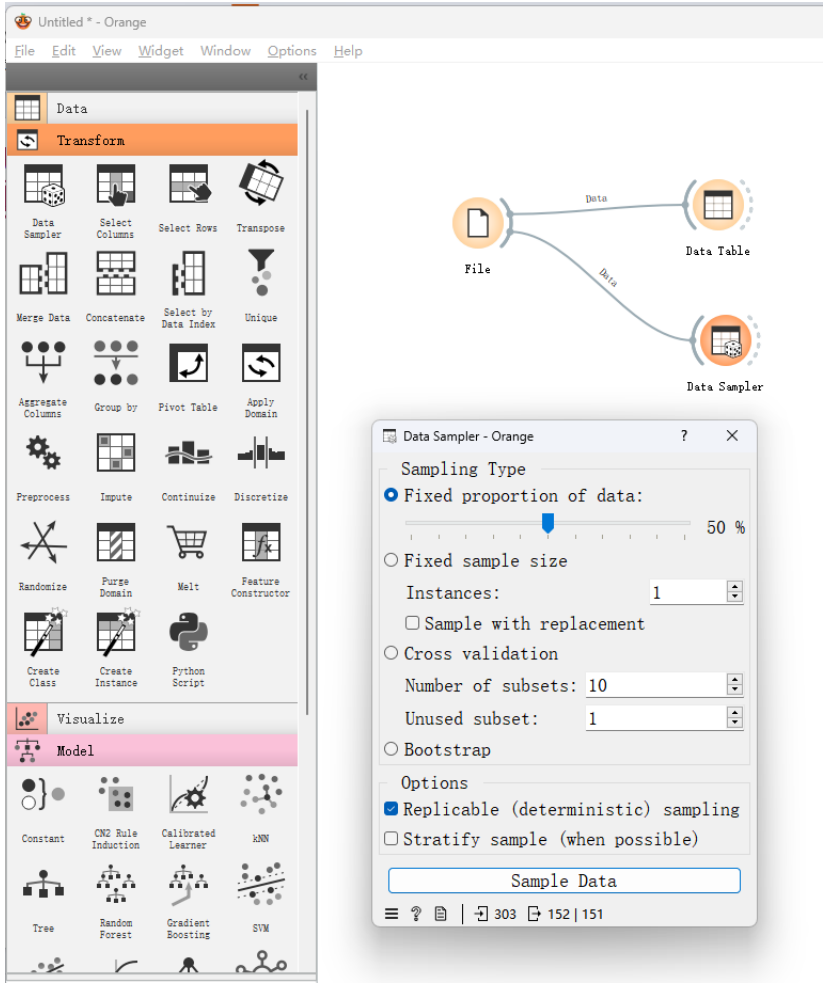
一次最小流程：从数据到一棵树

- **四步连线：File → Data Sampler → Tree → Prediction**
- 内置示例数据 heart_disease：303 条心脏病记录
- 拖出来、连上线，一棵决策树就训练完了
- 注意每个组件底部的「输入 / 输出」提示——Orange 不让你猜



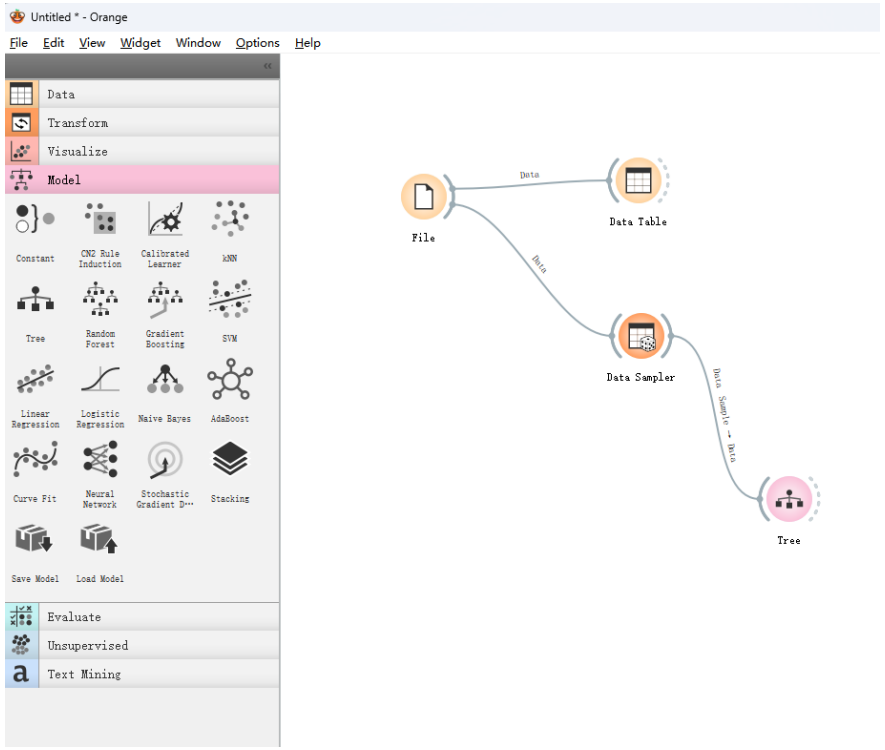
拆分数据：训练集与测试集（Data Sampler）

- **Data Sampler 按比例把数据切成两份**
- 看组件底部提示：303 → 152 | 152（两份各 152 条）
- 呼应上午「备考」比喻：一份当作业，一份当高考



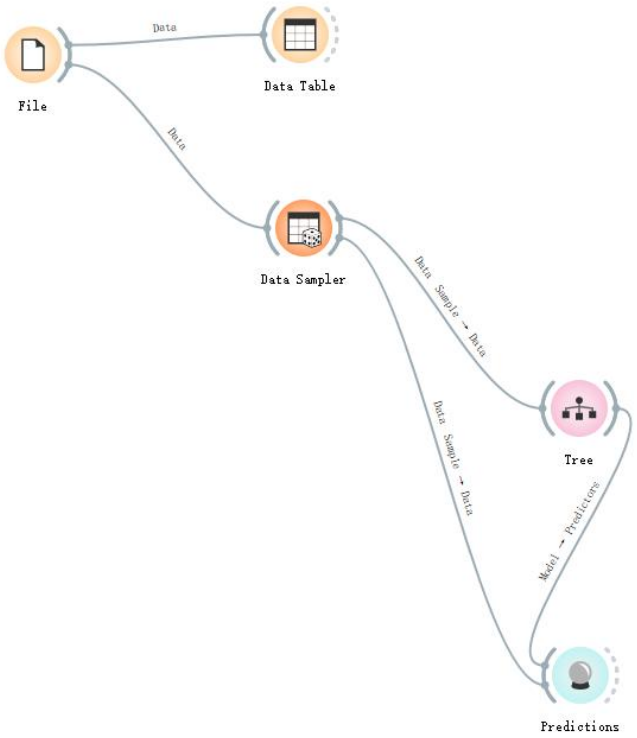
训练一棵决策树：双击 Tree 看它怎么长

- **把训练集接进 Tree 组件，训练自动完成**
- 双击打开：树长什么样、按什么特征分裂、剪枝参数全在这里
- 这正是下午「决策树原理」的预览：分裂、深度、叶子纯度



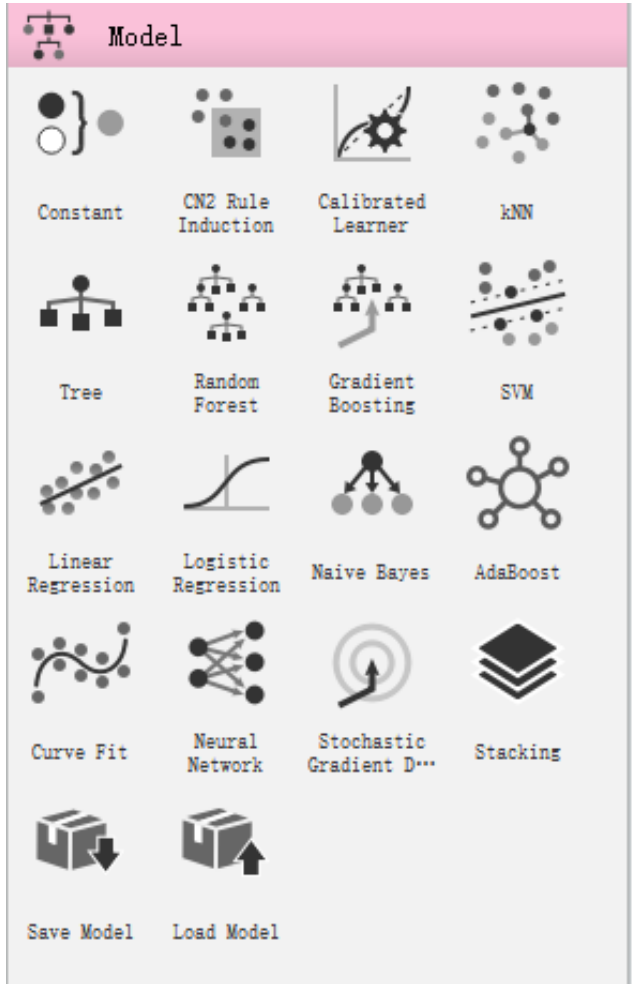
评估：Prediction 组件看模型准不准

- **Prediction：拿「没见过的数据」给模型打分**
- 连接 Data Sampler 的剩余部分 + Tree 模型
- 双击看预测结果表：对了几条、错了几条
- 呼应上午「测试集 = 高考」——Orange 帮你严格执行



换模型：把 Tree 换成 Forest，看看差别

- 复制评估流程，把 Tree 换 Random Forest / kNN
- 同一个测试集，看哪个模型更准
- Orange 让「模型比武」变成改一个组件的事



小结：Orange 与 WorkBuddy，两条路做同一件事

	Orange	WorkBuddy
方式	拖组件连线	提示词指挥
代码	零代码	AI 写代码
模型	点开即看（树/森林）	输出报告与图
适合	快速上手、看得见	全流程自动化
本课	下午实操主力	上午已演示

下午预告：Orange 里的树与聚类

- **下午用两个数据集实操：破产预测 + 财务舞弊**
- Orange 里做：决策树 → 看树 → 随机森林 → K-Means 聚类
- 现在休息，下午 14:30 开工

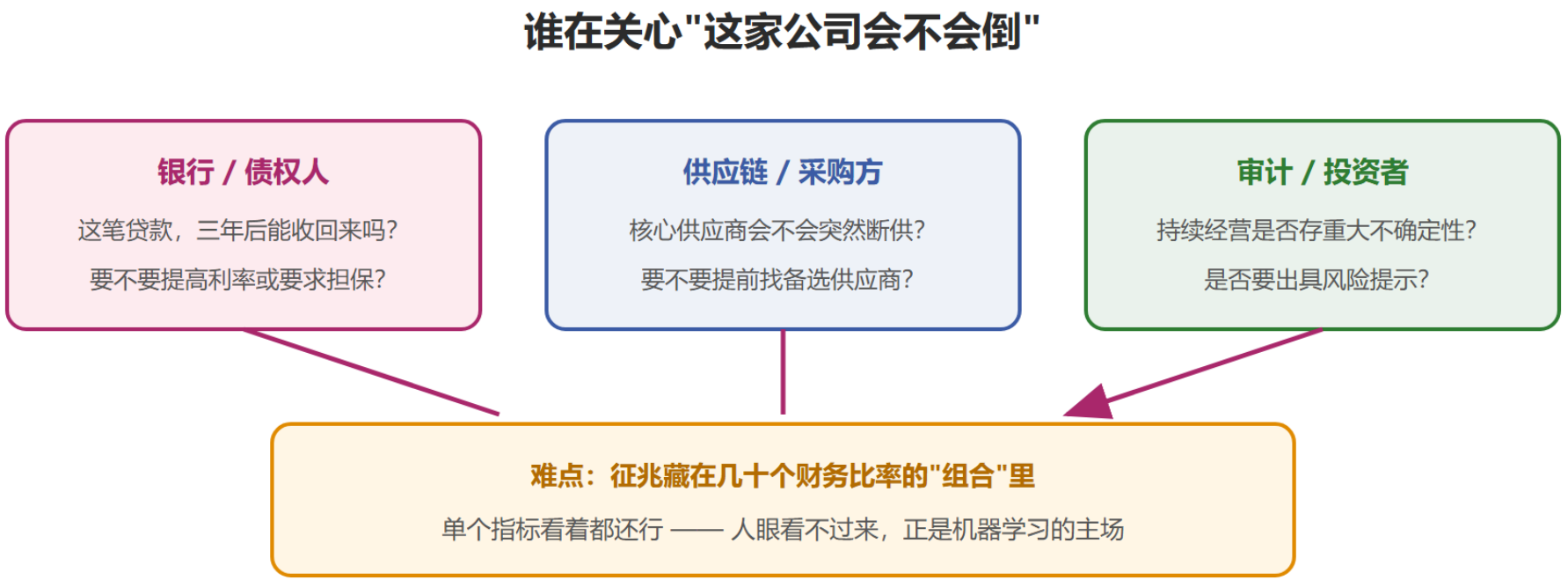
04

实战：基于AI Agent公司破产预测

搭项目 → 训模型 → 预测新公司

案例背景：为什么破产预测重要又困难

- 银行放贷、供应商授信、审计红旗——都想先知道"这家公司三年后还在吗"
- 征兆藏在几十个财务比率的组合里，人眼看不过来



学完这一节，你能带走什么

6,819	95	3.23%
家公司的真实财报	个财务比率指标	破产公司占比（罕见事件）

- **一套可复用的 AI 分析工作流：搭项目 → 清洗 → 训练 → 评估 → 预测新公司**
- 一套『指挥 AI』的提示词写法：目标、约束、指标、输出，一次说清
- 一双能挑错的眼睛：指标选得对不对、有没有数据泄漏、结果能不能信
- 一个能写进简历的项目：流程可复现、代码可交接、结论可解释

认识数据集

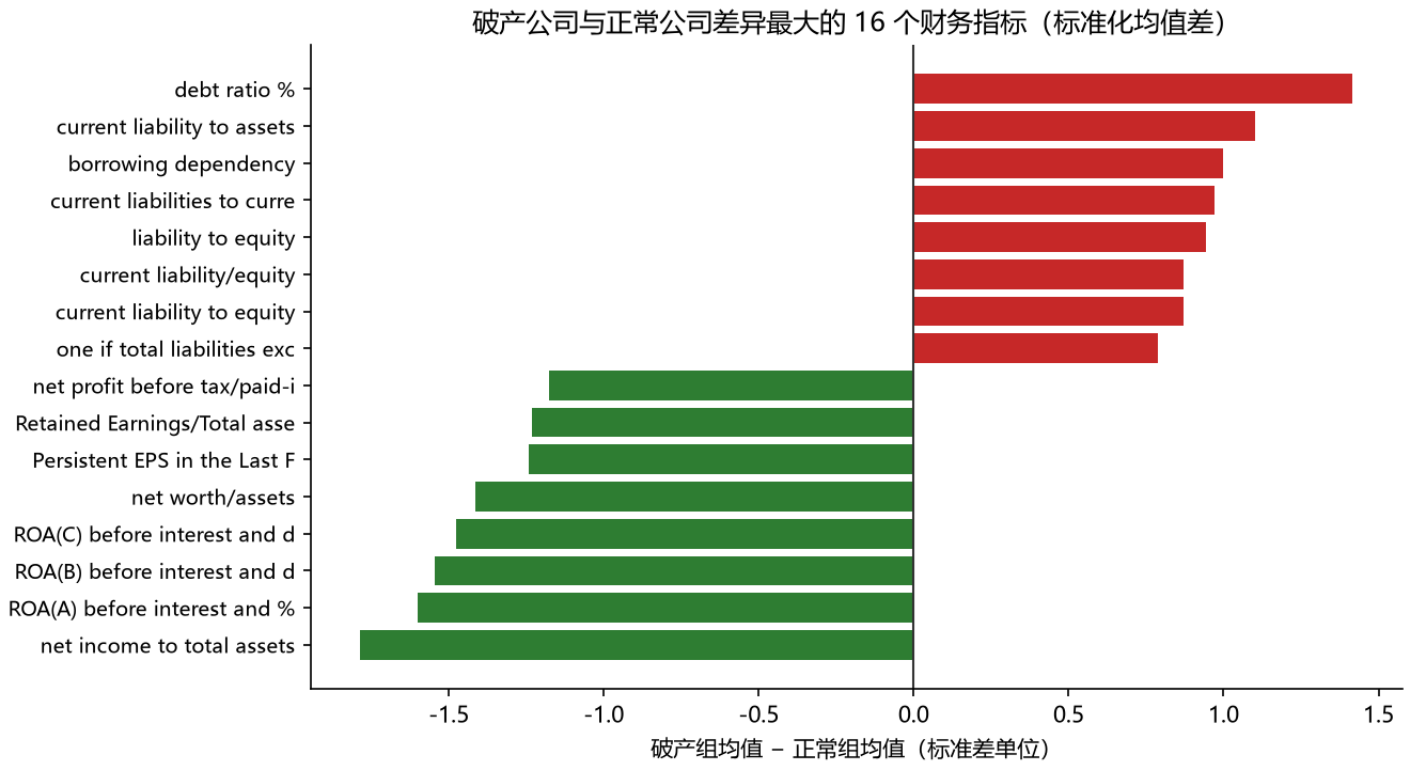
- **bankruptcy_data.csv：6,819 家公司 × 95 个财务比率 + 标签 Bankrupt?**
- 标签：1 = 破产（220 家，3.23%），0 = 正常（6,599 家）
- 来源：1999–2009 年《台湾经济杂志》；破产定义依台湾证券交易所规定

数据集样例：每行 = 一家公司的 95 个财务比率 + 破产标签（前 3 行正常，后 3 行破产）

是否破产	ROA(C)	负债率%	流动比率	净利润/总资产	借款依赖度
0.0	0.391	0.216	0.005	0.737	0.385
0.0	0.508	0.108	0.01	0.815	0.374
0.0	0.489	0.059	0.013	0.804	0.37
1.0	0.371	0.208	0.002	0.717	0.39
1.0	0.464	0.171	0.006	0.795	0.377
1.0	0.426	0.208	0.012	0.775	0.379

哪些指标最能区分破产公司？

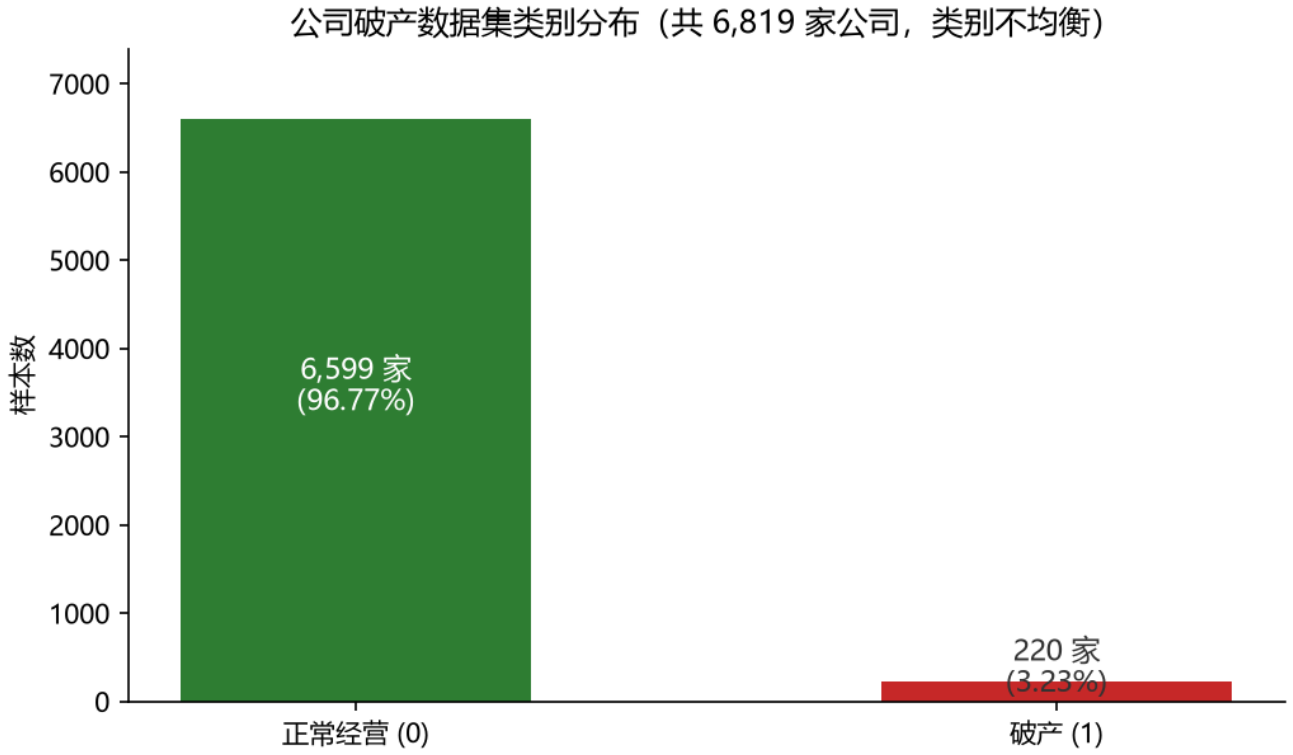
- 破产组：盈利类指标显著更低，负债类指标显著更高



关键挑战：类别不均衡与"准确率陷阱"

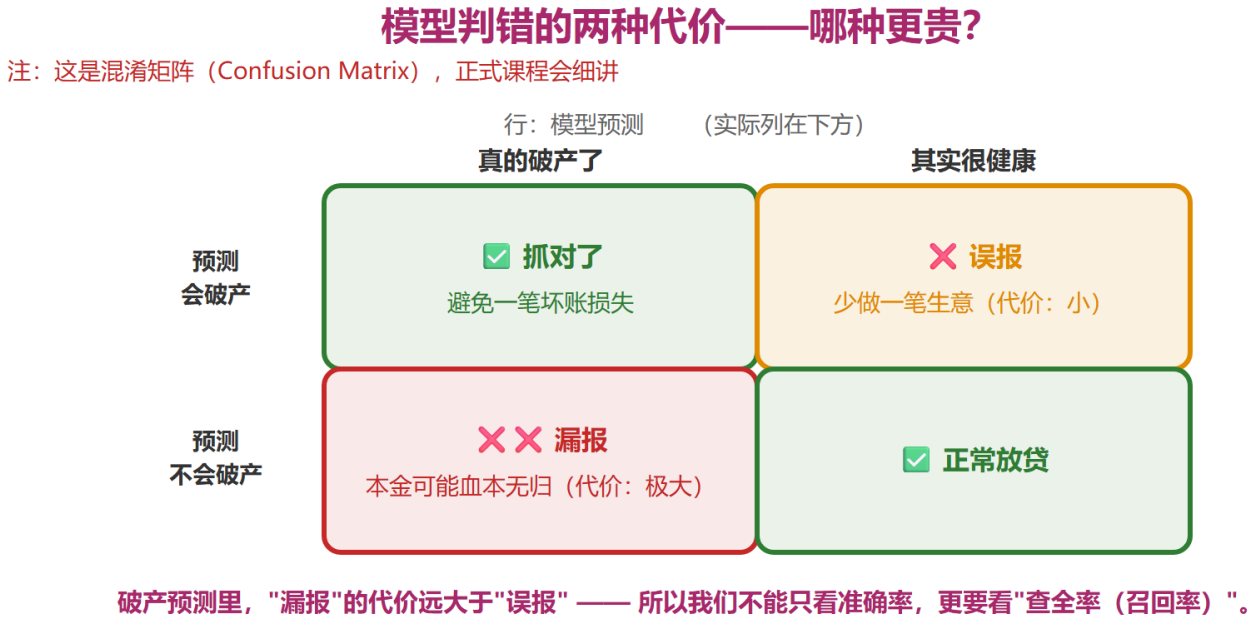
- **破产仅 220 家（3.23%）**
- 模型"无脑猜正常" → 准确率 96.77%，却一家破产公司都抓不到
- 在不均衡数据上，准确率是骗人的
- 换成：查全率 Recall / 查准率 Precision / F1 / AUC；并用类别权重或 SMOTE

📌 名词速览｜精确率/召回率/F1/AUC：就像从五个维度给『模型这个学生』打分——下一页图解，正式课程系统讲



判错的代价：哪种错误更贵？

- **漏报：真会破产的公司被判成『健康』 → 本金可能血本无归（代价极大）**
- 误报：健康公司被判成『高风险』 → 少做一笔生意（代价较小）
- 所以本课更看重『查全率（召回率）』，而不是准确率



名词速览①：评价指标（用『考试打分』来理解）

五个指标 = 给“模型”这个学生从五个维度打分

把模型看成“挑出可能破产公司的审查员”，把公司看成“学生”，把“是否真的破产”看成“是否真的不及格”

准确率 Accuracy

模型判对了的样本占全部的比例

→ **像：** *全判及格拿 99 分，却没用*

精确率 Precision

你点名的“差生”里真差生的比例

→ **像：** *点名的人里真不及格的比例（别冤枉好人）*

召回率 Recall

真差生里被你点出来的比例

→ **像：** *真不及格者中被点名的比例（别放过）*

F1 分数 F1-Score

精确率与召回率的“综合分”

→ **像：** *两科平均分，偏科会扣分（调和平均）*

AUC 曲线下面积

差生比好学生排得更靠前的概率

→ **像：** *抽 1 差生 1 好生，差生排前的概率*

本数据集里破产公司只占 3.23%，全猜“不会破产”准确率就有 96.77% —— 所以本课更看重召回率与 F1

这些指标的公式与更多指标，正式课程会详解

名词速览②：PR-AUC——不均衡数据上的精细标尺

为什么 AUC 在 3.23% 正类时可能“虚高”？

- AUC 由全部样本共同贡献：96.77% 的正常类把曲线“撑得很高”
- 它回答“正常和破产是否分得开”，看不出“到底抓没抓到破产”

PR-AUC 换了个视角

- 只看 Precision（查准）与 Recall（查全）的权衡曲线
- 全程只关注“破产”这一小撮正类抓得准不准
- 正类越稀少，PR-AUC 比 AUC 越灵敏

维度	AUC	PR-AUC
基线	0.50（固定）	= 正类占比 3.23%
受谁主导	多数类（正常）	正类（破产）
问的问题	两类分不分得开	正类抓得准不准

报数纪律：报 PR-AUC 必须同时报“正类占比基线”，否则数字没参照。

例：PR-AUC = 0.35（基线 0.032）→ 远超随机，才算真本事。

名词速览③：Brier 校准 + 两个不平衡开关

Brier Score：概率“靠不靠谱”

- 比较预测概率与真实结果：说 70% 破产，就应有约 70% 真的破产
- 取值 0（完美）~ 1（最差），越小越好
- AUC/PR-AUC 管“排序”，Brier 管“概率准不准”，两者不互相替代
- 校准曲线：把预测概率分箱，看各箱实际比例是否贴对角线

什么时候看它：

要拿“破产概率”去决策（如授信定价）时，概率必须准——这正是 Brier 的用武之地。

两个“不平衡”开关（写在提示词里）

stratif y（分层抽样）

划分训练/测试时保持正类占比一致，防止测试集恰好没抽到破产

class_weight='balanced'

训练时给少数类（破产）加权，模型不再“无脑倒向多数类”

一句话记住：

划分用 stratify，训练用 class_weight，评估看 PR-AUC + Brier——

这四件套都会出现在你们项目规则 PROJECT_RULES.md 里。

问题定义：这是哪种学习？

- **有标签 Bankrupt? → 有监督学习；离散输出 → 二分类**
- 代价不对称：漏报破产 ≫ 误报正常，所以更看重召回率
- 流程走 CRISP-DM：业务理解 → 数据理解 → 数据准备 → 建模 → 评估 → 部署
- 接下来：让 AI 从搭项目骨架开始，一步步被我们指挥

CRISP-DM：数据挖掘的六个标准阶段

最后一步：用模型预测『新公司』



我们今天从这里开始 → 再让 AI 搭骨架

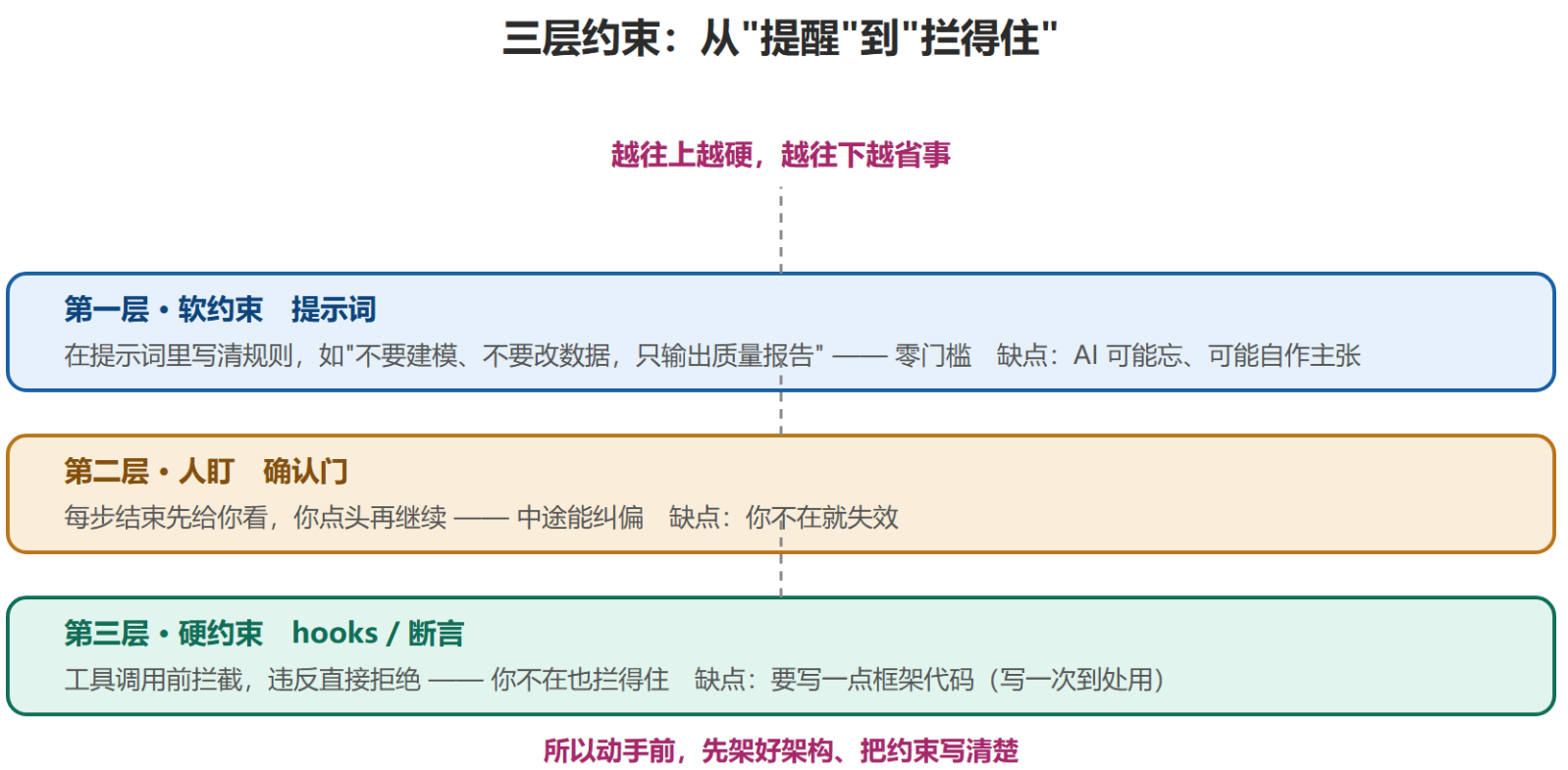
中间三步：下午演示的主战场

人机分工：谁拍板，谁干活



AI 负责『怎么做』，你负责『做什么』和『做到什么程度算好』——你不写代码，但你写规则

怎么让 AI 不敢乱来：三层约束



越往上越硬，越往下越省事 —— 所以动手前，先架好架构、把约束写清楚

顺序不能反：先架构 → 再约束 → 再动手

先立规矩，再动手

不立规矩

文件散落一地，找不到东西

命名全靠 AI 自由发挥

每步口径不一，结果对不上

改了什么没留痕，无法复现

跑完一次，下次从头再来

先立三件事

目录结构：东西放哪

命名规范：怎么起名

实验日志：改了什么留痕

三条写进 docs/README.md

后面每一步 AI 都照它执行

顺序不能反：先架构 → 再约束 → 再动手

下一步就照这个顺序来：先让 AI 搭骨架、定规矩，再开始干活

提示词第 0 步：先让 AI 当"项目经理"——搭骨架、定规矩

提示词（实测版）

我们要做一个『公司破产预测』机器学习项目。开工之前，请帮我完成两件事：

1. 建目录：当前目录下建 bankrupt_project/，含 data/、src/、models/、reports/、docs/；并把根目录的 bankruptcy_data.csv 复制一份到 bankrupt_project/data/ 作为原始数据（根目录原文件保持不动）。

2. 起草 docs/README.md：项目目标、目录职责、命名规范（模块 snake_case；模型『模型名_日期.pkl』；报告『YYYY-MM-DD_主题.md』；每次实验记录到 docs/experiment_log.md）。

先不要动手创建：把目录结构预览和 README 草案给我看，我确认后再创建。

项目骨架：开工前先让 AI 把"房子"盖好

推荐的项目目录结构

```
bankrupt_project/
    ├── data/  原始与处理后数据
    ├── src/  代码模块
    ├── models/  保存的模型
    ├── reports/  图表与分析报告
    └── docs/  项目文档与实验日志
```

关键文件：

docs/README.md —— 目标 · 目录职责 · 命名规范

docs/experiment_log.md —— 每次实验必记录

models/model_card.md —— 冠军模型的"身份证"

同时在 README 里定下的命名规范

- Python 模块：snake_case
  例：data_prep.py、train.py
- 模型文件：模型名_日期.pkl
  例：random_forest_20260914.pkl
- 报告文件：日期_主题.md
  例：2026-09-14_数据质量报告.md

为什么值得花这 3 分钟?

否则 AI 会把脚本、数据、图片随手散落，项目不可复现、不可交接

分工的落点：规矩写在哪两处

上一页的分工，要落到两个地方才管用：

① 项目规矩 → 写进 docs/README.md

- 目录职责、命名规范、实验日志格式
- 一次写清，后面每一步 AI 都照它执行

② 每步约束 → 写进提示词

- 「先看后做」「不许改原始数据」「调参只在验证集」
- 越具体，AI 越不容易自作主张

所以第 0 步不是建模，而是让 AI 当一回项目经理：搭骨架、定规矩

AI 做执行，人做判断——破产预测项目里的分工

人负责（你不可替代的部分）	AI 负责（交给它最划算的部分）
· 定义业务目标（什么叫"高风险"，多少算高）	· 读数据、写清洗与特征代码
· 判断代价：漏报 vs 误报哪个更贵	· 按你指定的协议跑 5 个模型
· 选择指标口径（看 F1 还是看召回率）	· 记录实验日志、画混淆矩阵与 PR 曲线
· 解读结果：这个特征为什么和破产有关	· 保存模型、生成新公司预测
· 决定阈值、决定是否上线	· 重复执行、批量执行、不喊累

提示词写得越清楚，AI 的执行越可靠；判断力，始终是你的核心竞争力。

模块化：按机器学习流程划分功能

模块	文件	产出
数据准备	`src/data_prep.py`	`train.csv` / `test.csv` + EDA 报告
特征工程	`src/features.py`	`features.csv`
模型训练	`src/train.py`	模型比武表 + 实验日志
模型评估	`src/evaluate.py`	混淆矩阵、PR 曲线报告
预测部署	`src/predict.py`	新公司预测结果

提示词第 1 步：数据理解（只看，不动手）

提示词（实测版）

bankruptcy_data.csv 共 6,819 行 × 96 列，首列 Bankrupt? 为标签。不要建模、不要改数据，只输出数据质量报告：

1. 列名是否有前后空格；
2. 有无零方差列（所有值相同）；
3. 有无内容完全相同的重复列；
4. 相关系数 |r| > 0.95 的高度共线特征组。

补充两点：标签列单独说明，不参与零方差和共线检查；相关系数可在全体样本上算（属探索性分析），但须注明——真正建模时若用到共线结论，要只用训练集重算。

① 列名与缺失情况

列名前后有空格：有（约 20 列）

缺失值：0

重复行：0

已统一列名格式 → 待你确认

② 异常列清单

零方差列：1 个

完全重复列：1 组（current liability...）

高共线组（|r|>0.95）：2 组

处理建议 → 待你确认后执行

这一页的教学价值

"数据永远没有想象中干净" —— 但只有把要求**说具体**（零方差 / 重复列 / 共线），

AI 才会去查这些细节，而不是泛泛回一句"数据质量良好"

提示词第 2 步：数据准备（逐条交代规则）

提示词（实测版）

拿到数据质量报告后，按下面五步做数据准备。每步先说明你要怎么做，再执行：

① 统一列名，去掉前后空格；

② 先按 7:3 **分层**划分训练集/测试集，`random_state=42`，存到 `data/` 下，命名 `train.csv` / `test.csv`，并在 `docs/experiment_log.md` 记录划分比例与正类占比；

③ 删零方差列（在训练集内判定）；

④ 每个共线特征组只保留一个（保留哪个由你判断，但要说明理由）——**分组必须只用训练集重算**：第 1 步那份质量报告里的共线组是在全样本上算的，直接拿来删列会把测试集信息渗进训练过程；

⑤ 标准化只用**训练集**的均值方差（`fit` 在训练集、`transform` 到两边），把 scaler 存到 `models/` 下供后面复用。

最后给我：清洗前后的特征数、训练集/测试集的形状与正类占比。

测试集从现在起锁死：训练、调参、选模型任何环节都不许读它，留到第 4 步只评估一次。

数据准备五条规则（全部写进提示词）

1	2	3	4	5
统一列名	删零方差列	去共线冗余	分层 70/30	只用训练集
去空格 / 小写下划线	信息量为零	每组留一个	保持破产比例	算均值和方差

第 5 条最容易被忽略，也最致命

如果标准化时用了"全体数据"的均值和方差，测试集的信息就悄悄溜进了训练过程

→ 测试成绩虚高 → 上线后一塌糊涂（数据泄漏）

提示词第 3 步：训练『一众』模型——比武大会

提示词（实测版）

训练 5 个候选模型（本课把它们当黑盒，训练流程完全一样）：逻辑回归、决策树、随机森林、梯度提升树（XGBoost / LightGBM）、SVM(RBF)。

带超参数的模型：先在训练集内部用 5 折交叉验证（验证集）调参，定参后再用整个训练集重训一次。

统一协议：分层 5 折 CV、类别权重 balanced、random_state=42，记录 F1/AUC/召回率到 experiment_log.md。

调参只能用验证集；测试集留到最后只评估一次。

输出『模型比武表』，按 F1 排序。

模型	准确率	查准率	查全率	F1	AUC
逻辑回归	—	—	—	—	—
决策树	—	—	—	—	—
随机森林	—	—	—	—	—
梯度提升树	—	—	—	—	—
SVM (RBF)	—	—	—	—	—

统一评估协议：分层 5 折 CV • 类别权重 balanced • random_state=42 • 全程未碰测试集

排序依据用 **F1**（不是准确率）——因为破产样本只占 3.23%

提示词第 4 步：挑冠军、最终评估、保存模型

提示词（实测版）

① 选出冠军并说明选型理由（不许只看准确率）；
② 调参只在验证集上做（5 折交叉验证），测试集始终不参与调参；
③ 只此一次在测试集上评估：混淆矩阵、召回率 / F1 / AUC、PR 曲线；
④ 模型连同列顺序与标准化参数保存为 models/best_model_<日期>.pkl，并写 models/model_card.md。

测试集混淆矩阵

预测↓ / 实际→

TN 真·正常 预测正常，实际正常	FN 漏报（最贵） 预测正常，实际破产
FP 误报 预测破产，实际正常	TP 抓对破产 预测破产，实际破产

本课重点：宁可多几个 FP，也要压低 FN

降低分类阈值会发生什么

默认阈值 0.50 → 判定"破产"

阈值 0.30 → 更敏感：查全率↑，查准率↓

阈值 0.70 → 更保守：查准率↑，查全率↓

业务决定阈值，不是模型决定

漏掉一家破产公司的损失 ≫ 多查几家
→ 我们倾向"低阈值、高查全率"

提示词第 5 步：无中生有一家"新公司"，让模型判生死

提示词（实测版）

参考训练集中正常公司的分布，随机生成一家新公司的全部财务指标；再生成第二家『危险版』（负债率调高、盈利调低）做压力测试。

两条约束：

- 指标之间不能自相矛盾（如：流动比率 ≥ 速动比率、负债率与净值/资产之和 ≈ 100%、每股指标与对应总额指标同向）。生成后请逐条自查并报告检查结果；
- 随机种子固定，保证可复现。

两者整理成与训练完全相同的列顺序与格式，加载保存的模型，输出：

1. 破产概率；2. 0/1 判定——同时给出默认 0.5 阈值和你自己选的业务阈值（并说明为什么选它）下的判定；3. Top5 影响特征（针对这一家的局部归因，不要用全局重要性）；4. 一段业务解读。

新公司 A（按正常公司分布生成）

ROA(C)：在健康区间随机

负债率 %：中等水平

流动比率：> 1

破产概率：低 → 判定：正常

新公司 B（危险版压力测试）

ROA(C)：下调（接近破产组均值）

负债率 %：上调（接近破产组均值）

流动比率：< 1

破产概率：高 → 判定：高风险

这一步最容易翻车的两个细节（必须写进提示词）

① 新数据的**列顺序、列名、标准化参数**必须与训练时完全一致 → 否则"张冠李戴"

② 生成指标时要保证**业务自洽**（负债率高时 ROA 通常不会同时很高）

好提示词 vs 坏提示词：复盘对照

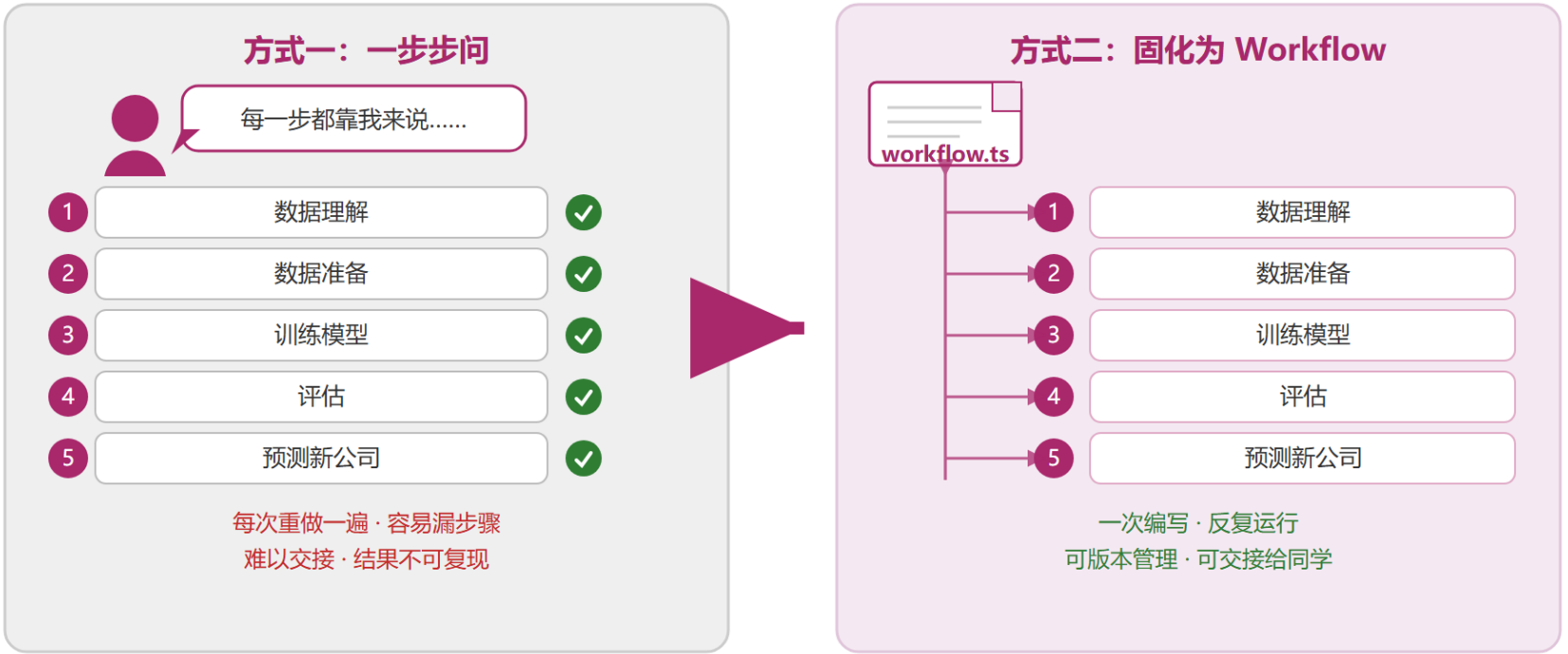
❌ 模糊	✅ 精准
"帮我预测哪些公司会破产"	"Bankrupt? 为标签，做二分类"
上来就建模型	先搭目录、定命名规范，确认再开工
没提数据质量	查零方差/重复/共线列，先出报告
单模型碰运气	5 模型 + 统一 CV 协议 + 实验日志

今天跑通了流程，但还有四块没讲透

- **今天我们已经把整条链路跑通了：搭项目 → 清洗 → 训练 → 评估 → 预测新公司**
- 提示词会给大家带走，照着做，你能复现今天的结果
- 但提示词里有不少细节，今天只用了、没讲透：
- 数据获取：数据从哪来、口径怎么判、怎么验收
- 数据处理：缺失值与异常怎么处理、特征为什么这样构造
- 数据可视化：怎么把结果讲成一个能被相信的故事
- 模型评估：除 F1/AUC 外还要看什么、阈值怎么按成本定
- **所以自己从头组织一套提示词，仍然有难度——这正是正式课程要补的**

从『一步步问』到『固化成流程』

- **前面我们一步一步用提示词指挥 AI：清晰可控，但每次都要重来一遍**
- 熟练之后，就该把流程固化：一次编写，反复运行
- 固化后的好处：可复现、可交接、可版本管理、能交给同学一键重跑
- 做法：把每一步的提示词整理进 docs/，再用代码把它们串起来



正式课程：三天，串起一条完整技能链

- **Day 1 数据采集：驱动 AI 从公开渠道采集年报、公告、研报，并学会验收**
- Day 2 机器学习建模：从基础模型到集成方法，验证设计与评估指标一次讲透
- Day 3 文本挖掘：从年报、研报文本中衡量公司信号与指标，可视化并交付报告
- 例如：从财报与研报里抽取风险表述、语气变化、主题分布，做成可入表的指标
- 贯穿三天的是「人在环中」训练：AI 负责跑，你负责判断
- 带走：一份报告 + 可复现 GitHub 仓库 + 公开项目主页 + 一套复用模板



把今天的五步写成一个 Workflow

workflow.ts（实测版节选）

```
type Step = { id; label; file; cwd; tools; out; lock?; append? };

const STEPS: Step[] = [
  { id:'0_scaffold', file:'0_搭骨架.md', tools:READ_WRITE,
    out:['bankrupt_project/data/','.../src/','.../docs/README.md'] },
  { id:'1_data_understanding', file:'1_数据理解.md',
    tools:[...READ_WRITE,'Bash'], out:['reports/*.md'] },
  { id:'2_data_prep', file:'2_数据准备.md',
    out:['data/train.csv','data/test.csv'] },
  { id:'3_train_models', file:'3_模型比武.md',
    out:['docs/experiment_log.md'],
    lock:['re:data[\\/]test','re:test[\\w*-]*\\.csv'] }, // 训练步禁碰测试集
  { id:'4_evaluate_champion', file:'4_冠军与终评.md',
    out:['models/model_card.md'] },
  { id:'5_predict_new_company', file:'5_新公司预测.md',
    out:['reports/*.md'] },
];

for (const step of STEPS) {
  const prompt = readFileSync(join(PROMPT_DIR, step.file),'utf8');
  for await (const m of query({ prompt, options:{ cwd, allowedTools:
step.tools,
                                                   hooks: gateHooks(step.lock)
} })) {...}
  const miss = missing(step.cwd, step.out, t0);  // 产物校验（含 mtime）
  if (miss.length) break;                        // 缺失即停
}
```

- **6 条提示词各存成一个 .md 文件，脚本按序读取**
- 每步声明 tools（工具白名单）与 out（产物清单）
- 每步独立会话执行，互不污染上下文
- 支持 WORKFLOW_ONLY=0,1,2 只跑指定步骤
- 全程写 workflow.log，后台也能看进度

Workflow：把你的判断变成机器可执行的断言



流水线里没有人，所以要把『人在回路中的确认』换成断言

门控机制：让模型不可能偷看测试集

① 工具白名单：每步 allowedTools 限定它能用哪些工具

② 测试集锁定：第 2 步划分完就锁死 test.csv，第 3、4 步挂 hooks

③ 正则而非写死路径：锁 data[\\/]test 与 test*.csv 的所有变体

——连 AI 自己派生的 test_scaled.csv 也拦得住

④ 产物校验：每步声明 out，跑完用 mtime 检查是否本步新产出

⑤ 失败即停：任何一步产物缺失，后续步骤不再执行

门控代码（节选）

```
// 门控钩子：拦截一切提到被锁文件的工具调用
function makeGate(lock: string[]) {
  const matchers = lock.map(p =>
    p.startsWith('re:') ? new RegExp(p.slice(3))
                        : { test:(s:string)=>s.includes(p) });
  return async (input:any) => {
    const raw = JSON.stringify(input.tool_input ?? {});
    const hit = lock.find((p,i) => matchers[i].test(raw));
    if (hit && ['Read','Bash','Grep','Glob','Edit','Write']
               .includes(input.tool_name)) {
      return { hookSpecificOutput: {
        hookEventName:'PreToolUse', permissionDecision:'deny',
        permissionDecisionReason:
          `【门控拦截】本步禁止触碰测试集（${hit}）` } };
    } return {}; };
}
options.hooks = { PreToolUse: [{ matcher:'*',
hooks:[makeGate(step.lock)] }] };

// 产物校验：存在 & 是本步之后新写/改写的（mtime）
const miss = missing(step.cwd, step.out, t0);
if (miss.length) break;
```

对你的价值，以及怎么上手

框架一劳永逸：目录、命名、提示词、门控固定之后，你只改模型和参数

- 换随机森林→XGBoost、调阈值、加特征，重跑一条命令就有新结果
- 门控保证无论怎么改，测试集都只在第 4 步被评估一次——结果可比、不自欺
- 对个人：一次搭建，每个数据集都能重跑；对团队：流程进 Git，同学可一键复现
- 上手路径：6 条提示词存进 docs/prompts/ → workflow.ts 串联 → 加 hooks 门控
- 提醒：并行会放大错误——每个子任务先单独跑通，再开启并行

05

总结与答疑

回顾 + 课后任务

一页回顾

- 机器学习 = 从经验 E 学，完成任务 T，用性能 P 衡量（Mitchell，1997）
- 三大范式：有监督 / 无监督 / 半监督
- 破产预测 = 有监督二分类 + 类别不均衡 → 指标选对才有意义
- 驾驭 AI：先立项目规矩 + 精准描述 + 确认门 + 统一评估协议
- 从提示词到 Workflow：把五步固化成可复现的脚本
- 先导课定位：流程跑通了，原理与判断力留给正式课程（Day 2 全貌 + Day 1/3）



今天上午，你带走的两样东西

90 分钟，你带走的两样东西

判断力（前半场）

看得懂模型到底在学什么

分得清三种范式

知道训练 / 测试集为何必须分开

看得出准确率在骗人

明白代价不对称只能人定

这是你管 AI 的资格证

驾驭力（后半场）

会用提示词给 AI 立规矩

会用确认门中途纠偏

知道 hooks 能硬约束

能把流程固化成 Workflow

拿到了可复现的项目骨架

这是你落地执行的手

全程 0 行手写代码

两样加起来，才是『会用 AI 做机器学习』——而不只是『看过别人做』